

SkillC: architecture, cases, and integration evidence

Supplementary material | Read alongside MLADS_AGENT_Submission.docx

S1. What is carried over from the source paper

This supplement restores the source paper's substantive figures, cases, tables, and theoretical foundations while keeping implementation central. It keeps three evidence types separate: the original pack-level study, later repository experiments, and a proposed Microsoft Agent Framework adapter. No later experiment is attributed to the original paper.

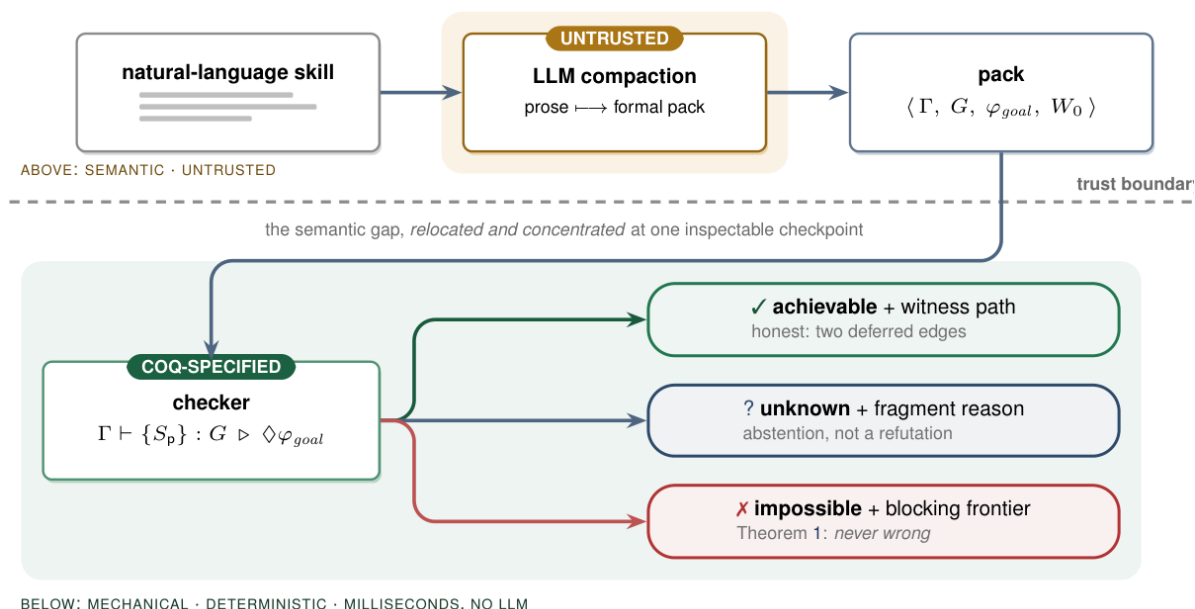


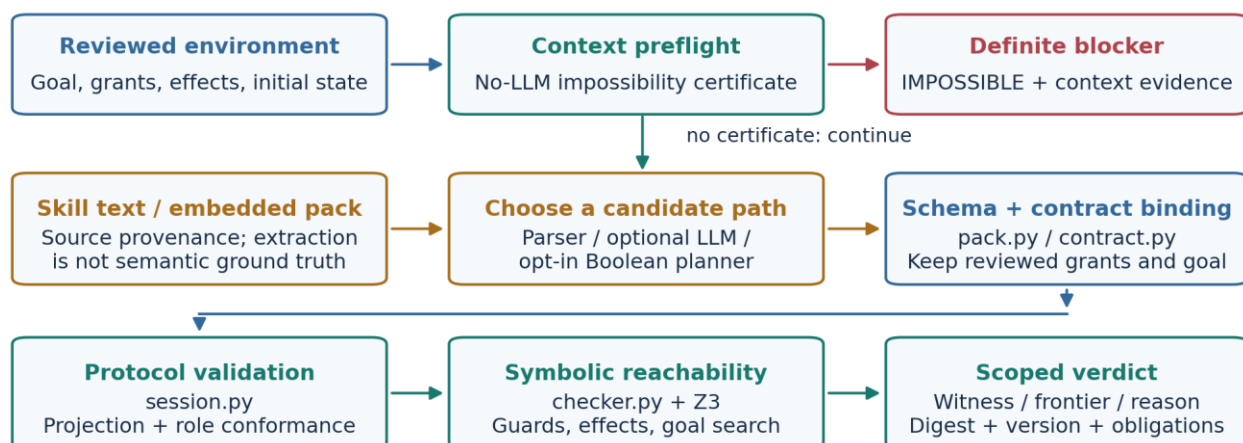
Figure S1. Original Figure 1 extracted from our main_submission.pdf, page 2. Its pack-relative guarantee must not be read as end-to-end mechanization of the executable checker.

Source-paper item	Where it is used in the MLADS package
Trust-boundary figure (p. 2)	Redrawn in main Figure 1; original above.
Flight example (pp. 3-4)	S3 capability table and witness.
Validation signals (p. 4)	Main Table 1; implementation interpretation in S2.
15-case matrix and failure modes (p. 5)	S8 matrix; S3 worked cases.
Six-case extension; theory (pp. 6, 13-18)	S8 extension; main Formal foundation and S9-S10 proof scope.
Partition of obligations (p. 18)	Updated implementation-status table in S7.
Token-economics table (p. 19)	S5, explicitly modeled rather than measured.

A pack P comprises capabilities, global protocol, goal, and initial world. Capabilities carry guards and Boolean/numeric effects. A protocol goal marker means 'the goal holds here', not 'eventually do this'. Schema acceptance does not certify intent fidelity or actual backend effects. An abstract witness is conditional, and a refutation has an explicit scope.

S2. How the implementation is assembled

IMPLEMENTED | SkillC compiler and admission components



ACHIEVABLE is conditional | IMPOSSIBLE has a scope | UNKNOWN is an abstention

Figure S2. Implemented compiler/admission components; paths are chosen explicitly.

Module / surface	Responsibility and boundary
frontend\markdown.py	Embedded packs, supported prose extraction, source provenance; tool-only fallback is not a semantic goal.
frontend\llm.py	Optional Azure OpenAI or Anthropic extraction. The provider output remains untrusted.
frontend\contract.py	Bind reviewed goal, grants/effects and initial context while retaining the candidate protocol.
pack.py	Schema validation and normalization; reject malformed packs explicitly.
planning.py	Bounded Boolean breadth-first search for one role; candidates go through the normal checker.
session.py / checker.py	Projection/conformance plus guarded symbolic reachability and Z3 queries.
cli.py / Verdict	Machine-readable identity, scope, witness/frontier and obligations; distinct exit codes.

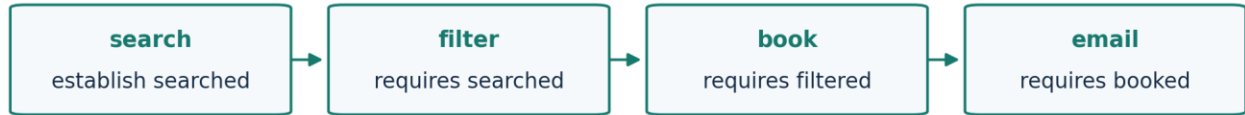
Ordering matters. The goal-only CLI path first builds a context-only pack from the reviewed contract. A capability-context certificate can end the request before LLM compaction. If no such certificate exists, the caller may request supported deterministic synthesis or use extraction. Context preflight is conservative: its failure to refute is not an admission.

Positive planning is deliberately narrow. The planner respects guards and delete effects, uses a visited state set and a default 5,000-state limit, and returns an explicit abstention for unsupported multi-role/numeric synthesis or exhausted/limited search. It proposes a new protocol; it does not retroactively certify the source's original protocol. The checker itself supports a broader fragment.

Default scope validates a declared protocol. Goal scope must not turn NON_PROJECTABLE or NON_CONFORMANT into a universal business-goal claim. If no independent context refutation is available, that rejection becomes UNKNOWN/PROTOCOL_ONLY.

S3. Worked cases from the paper and repository

FLIGHT CASE | Goal: booked AND confirmation_sent



Without email: booking may succeed, but the full goal cannot.

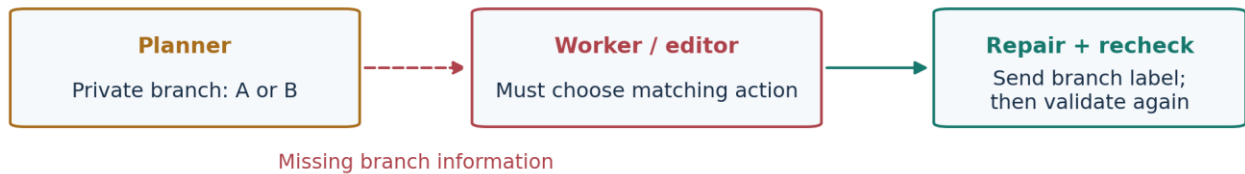
With email: the four-step sequence is an abstract witness, not a payload guarantee.

Figure S3. Our flight example, translated into an implementation walkthrough.

Capability	Guard	Fact established
search	true	searched
filter	searched	filtered
book	filtered	booked
email	booked	confirmation_sent

Start with none of these facts true. Under the declared frame assumption, a context containing search, filter, and book can reach booked but has no producer of confirmation_sent. The full conjunction is impossible in that context. Adding email enables search -> filter -> book -> email. The actual sending of the message remains a runtime obligation; the checker cannot infer delivery from an unreliable tool declaration.

COORDINATION CASE | Available tools do not guarantee a valid handoff



NON_PROJECTABLE refutes the declared protocol; another protocol may still achieve the goal.

Figure S4. Unobserved-choice failure and protocol repair, adapted from the paper's coordination case.

The paper also motivates a handoff where both parties wait for an incoming message, so neither can begin. More retries cannot create the absent send. These examples illustrate declared-protocol failure, not evidence that every alternative protocol fails.

Additional case	What the result means	Practical response
Budget constraint	Every declared path violates the goal's numeric refinement.	Review a real capability or constraint; do not silently weaken the goal.
Permanently blocked guard	The action prerequisite cannot be established.	Fix the prerequisite or abstain; increasing retries is not a repair.
Intent / payload mismatch	A pack may be admitted while the real task fails.	Human intent review and independent runtime postconditions.

S4. Live-backend benchmark: full comparison

Each task has four profiles and one trial per configuration/profile. The gate and ungated agent share the same runtime prompt; the LLM-compaction configuration adds extraction, while the direct executor also replaces action-selection inference. Its lower token count must not be presented as admission-only savings. The backend effects are executed, not merely simulated.

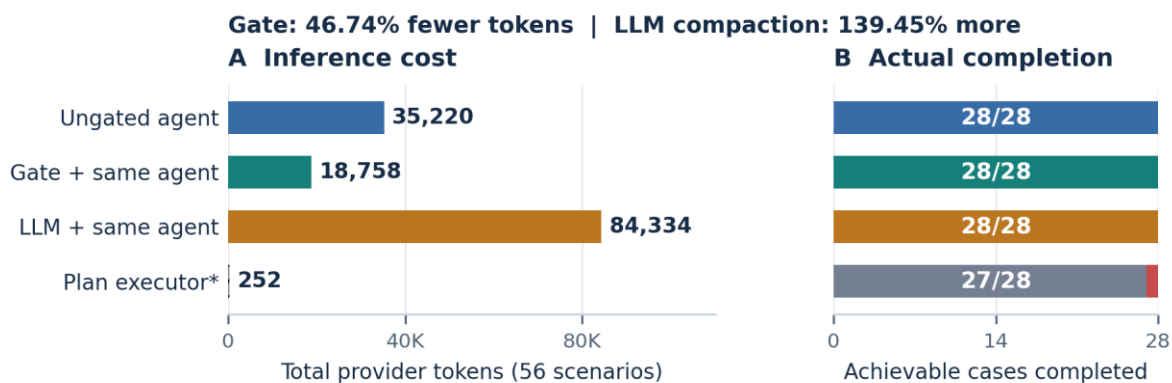


Figure S5. Measured total tokens and completed achievable cases; all 28 negatives are rejected in every SkillC configuration, with no false rejections in this bounded matrix.

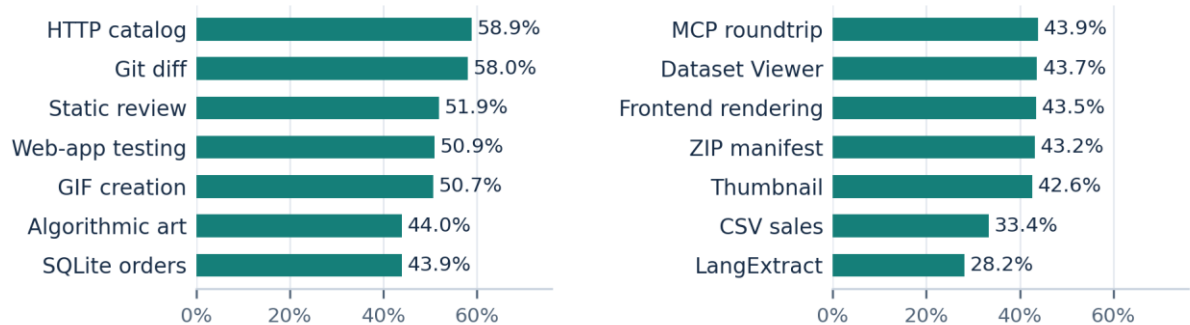
Task (4 profiles)	Ungated	Gate + agent	LLM + agent	Plan
CSV sales	1,888	1,258	4,041	0
SQLite orders	2,251	1,262	4,042	0
Git diff	2,982	1,252	4,032	0
ZIP manifest	2,197	1,248	4,020	0
Thumbnail	2,192	1,258	4,031	0
HTTP catalog	3,105	1,276	4,066	0
Web-app testing	2,809	1,380	5,926	0
Frontend rendering	2,517	1,422	7,502	0
GIF creation	2,920	1,440	8,196	0
Algorithmic art	2,548	1,428	12,530	0
MCP roundtrip	2,510	1,408	8,245	0
Dataset Viewer	2,645	1,490	6,570	0
Static review	2,991	1,440	7,118	0
LangExtract	1,665	1,196	4,015	252
Total	35,220	18,758	84,334	252

Read the denominator. All families have 2/2 achievable successes in each of the first three configurations. The direct executor has 1/2 in LangExtract and 2/2 elsewhere. Missing operations/inputs are enforced in the adapter environment. Task diversity is not equivalent to protocol-complexity diversity.

S5. Per-family results and original cost model

Token reduction by task family: deterministic gate vs ungated

Four environments per family; with and without SkillC, 2/2 achievable cases complete.



Observed single-pass comparisons, not confidence intervals or a production distribution.

Figure S6. Per-family measured reduction. Overall savings use summed tokens: $(35,220 - 18,758) / 35,220 = 46.74\%$, not an average of family percentages.



Figure S7. Supporting evidence sets are separate: profile compatibility, semantic mutation detection, the curated corpus, and checked planning replay.

Table S1. Our original Appendix E estimates, NOT live-backend measurements.

Reason	Turns: low/typ/high	Agents	Typical waste	Leverage
MISSING_CAPABILITY	3 / 8 / 14	1	50,240	18x
GOAL_UNSAT	8 / 18 / 30	1	176,040	63x
NON_CONFORMANT	6 / 15 / 30	2	261,900	94x
NON_PROJECTABLE	6 / 15 / 30	2	261,900	94x
BLOCKED_GUARD	12 / 25 / 50	1	305,750	110x

The source model assumes a 2,500-token harness, about 1,700 skill tokens, 700 appended tokens per turn, and 250 output tokens per turn. Its growing-context input formula is $T(S + K) + gT(T - 1)/2$, plus T_o output. Roughly 2,800 compaction tokens give the per-mode leverage above. These estimates price counterfactual avoided runs.

For the seven structural refutations in the original 15-spec corpus, Appendix E reports 12.2K compaction tokens versus about 1.07M modeled avoided tokens (0.28M-3.15M band), or about 87x leverage. This is not the measured 46.74% live result. UNKNOWN prevents no run by itself, and zero checker tokens does not mean zero CPU or integration cost.

S6. Proposed Microsoft Agent Framework adapter

The repository contains an integration opportunity, not an implemented MAF adapter: there is no `agent_framework` import or MAF middleware implementation in the inspected application sources. The Azure OpenAI and MCP benchmark calls do not establish MAF integration. The following is a source-informed engineering design to implement and test.

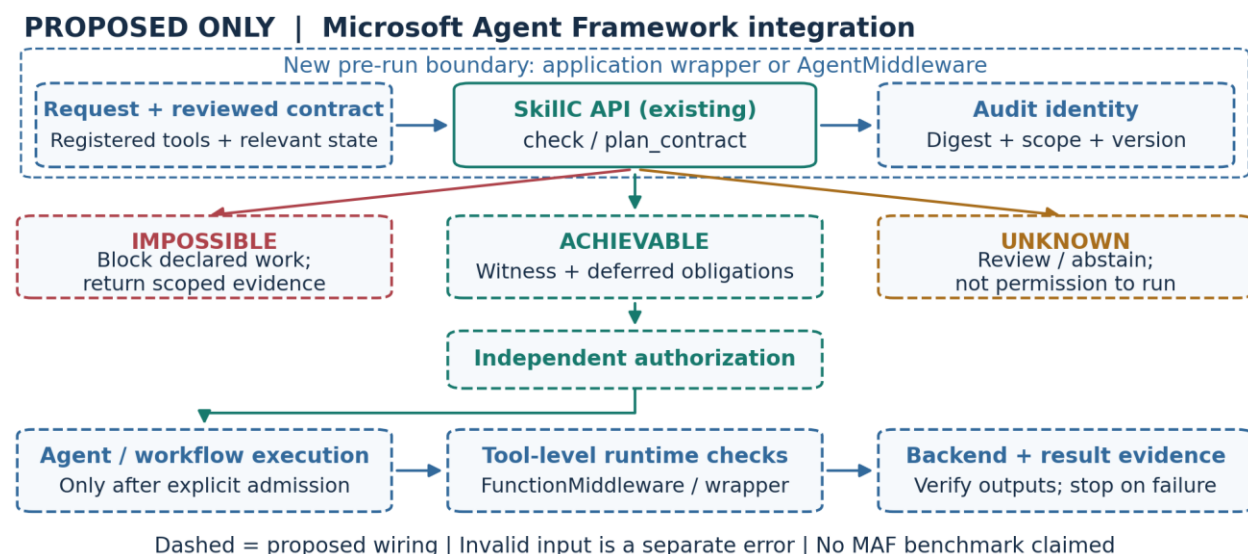


Figure S8. Proposed runtime integration. Solid SkillC core is reusable; dashed adapter and runtime-integration components need implementation and validation.

Official extension point	Proposed SkillC mapping
AgentMiddleware	Run-wide admission before the continuation invokes the agent. Emit explicit blocked/review results, not successful empty output.
FunctionMiddleware	Validate the permitted invocation and inspect the real tool result; propagate postcondition failures rather than returning success.
Executor / WorkflowBuilder	An initial typed gate routes admitted, rejected, and review-required outcomes to separate destinations.
MCPStdioTool / MCPStreamableHTTPTool	Bind discovered tools to reviewed contracts; inventory/schema discovery is not semantic effect validation.
configure_otel_providers	Correlate application-owned verdict records with traces; do not assume metadata automatically propagates between contexts.

Contract acquisition. Resolve an invocation-specific contract from application-owned configuration and observed tool/state facts. A tool-list match can establish inventory consistency, but an MCP schema does not prove that calling a tool has the declared business effect. Bind the contract to a reviewed/generated pack and perform context preflight before model use.

Invocation identity. If admitted, pass the same approved context into execution and retain the pack digest, checker version, decision/refutation scope, witness, and deferred obligations in a separate audit record. Do not log secrets or unnecessary prompt/payload contents. Changes to the goal, allowed tools, or relevant state invalidate admission. Long-running workflows need a new checkpoint after such changes, not one reusable global approval.

S7. Admission policy, runtime obligations, and tests

The adapter is a policy boundary, not just a decorator around a model call. Its response must be typed and observable: blocked work, review-required work, or admitted work. Do not convert errors or abstentions into a successful-looking empty answer.

Version boundary. Pin and test the SDK before implementing callbacks: current conceptual examples use AgentContext while API-reference pages also expose AgentRunContext. Do not mix their signatures. Test both streaming and non-streaming behavior, and explicitly propagate application audit identity rather than assuming one middleware context is shared everywhere.

Condition	Required integration behavior
IMPOSSIBLE: capability context	Stop before inference; return the independent blocking certificate.
IMPOSSIBLE: declared protocol	Block that workflow; explicitly avoid claiming every alternative goal plan is impossible.
ACHIEVABLE	Continue only if independent authorization/approval also succeeds; retain runtime obligations.
UNKNOWN / unsupported / timeout	Route to explicit review or a separately authorized fallback; no default permission.
Malformed pack / contract mismatch	Surface an input/integration error; do not coerce it into IMPOSSIBLE or ACHIEVABLE.
Tool/state drift	Invalidate the admission and revalidate the changed context.
Runtime postcondition failure	Stop/escalate; do not claim success. Postchecks cannot undo completed side effects; pre-authorize consequential tools.

Table S2. Our obligation partition, updated with current implementation status.

Obligation	Where it belongs	Current status
Goal achievability	Static validation of declared pack	Implemented checker and bounded studies.
Per-step safety / privilege	Independent authorization and policy	Not established by an achievability verdict.
Payload faithfulness	Runtime verification	Task-specific benchmark monitors; not a general MAF safety layer.
Intent fidelity	Human review of task/pack	Still a semantic trust obligation.

Table S3. Acceptance tests to run before claiming MAF integration (not results already obtained).

Test	Required observation
Missing essential operation	No agent/model invocation; recorded scoped blocking reason.
Missing optional diagnostics	Achievable business goal is not rejected merely because optional work is unavailable.
Unsupported synthesis / timeout	Review-required outcome; no silent admission.
Empty / incomplete / misaligned extraction	Postcondition failure, including empty-list regression; no false success.
Tool grant or goal changed after admission	Stale result is invalidated; changed context is checked again.
Streaming and workflow branches	Rejection emits no model stream; each fan-out path has the intended scope and policy.

S8. Evidence provenance and reproducibility

The figures and per-task table aggregate saved results rather than rerunning paid model calls. The main MCP matrix is stored in `runs\20260921_133023Z_live_backends`; the primary LangExtract result is `runs\20260921_langextract_live\postcondition_checked`. The earlier LangExtract diagnostic run remains in the measured subdirectory and is not silently included in the primary four-configuration comparison.

Evidence	Accounting / interpretation
MCP run	52 scenarios; 260 initialized sessions including references; 331 tool calls; 227 Azure calls; 131,436 tokens.
Corrected LangExtract	4 scenarios x 4 configurations; 24 Azure calls; 7,128 tokens, including nested inference.
Primary combined comparison	56 scenarios per configuration; 251 calls overall; 138,564 tokens in total.
Retained LangExtract diagnostic	7,563 additional tokens; excluded from the main comparison, included in total experiment cost.
Total including diagnostic	146,127 tokens and 277 distinct calls; reused CSV/SQLite ledgers counted once.
What is not established	Production prevalence, general compaction accuracy, MAF performance, or measured multi-agent protocol-failure savings.

The benchmark verifier reconciles provider usage, request hashes, discovered tools, saved trial outcomes, and actual artifacts. The MCP restrictions are physically enforced; labels are relative to the authored adapter contracts. Five catalogued cloud/Aspire workflows remain unexecuted rather than being replaced by mock results. The primary trials use real backends but are not unchanged production skill workflows.

Reproduction entry points are `benchmark_live_backends.py` / `verify_live_backends.py` and `benchmark_langextract_live.py` / `verify_langextract_live.py` under `scripts`. Use a fresh run directory for new live trials and the optional LangExtract environment. Keep credentials out of manuscripts and artifacts. The canonical source for the original paper is `main_submission.pdf`; its LaTeX and PDF are not modified by this adaptation.

Source notes

The related-work bibliography is in `MLADS_AGENT_Submission.docx`, after its four content pages.

Our benchmark records are experiment provenance, not bibliography entries. The live study, correction report and token-economics evidence guide under `docs`, together with the saved run ledgers, identify the inputs and accounting. S11-S12 extends this audit to the earlier experiments and commit-pinned historical data.

API details and version caveats: `docs\MICROSOFT_AGENT_FRAMEWORK_INTEGRATION_RESEARCH.md`. The integration and its performance remain untested.

Original 15-pack corpus: positive class = `ACHIEVABLE`.

Ground truth	Predicted achievable	Predicted impossible
Actually achievable	6	0
Actually impossible	2	7

Six achievable controls and seven structural failures are classified correctly; two planted false admissions expose intent/payload gaps. The separate six-case extension returns two `ACHIEVABLE`, three `IMPOSSIBLE`, and one `UNKNOWN`; it is not pooled into this matrix.

S9. Formal model and principal guarantees

The following restates our theoretical results from the original manuscript's Appendices A-D, rather than introducing new theorems. The original numbering is retained. These results explain the static-validation boundary behind the implementation.

Model and abstraction

A pack $P = (\Gamma, G, \phi_goal, W_0)$ fixes a capability context, global protocol, goal and initial world. Each capability has a guard and Boolean/numeric effects. Configurations pair residual protocol with world state. Achievability asks for a declared run reaching a goal-satisfying configuration; a goal marker checks the world at that point, not an eventual promise. Protocol realizability and role conformance are separate premises, not consequences of possessing the right tools.

Let a map concrete configurations to abstract configurations. STEP-SIM requires every concrete transition $C \rightarrow C'$ to have a corresponding abstract transition $a(C) \rightarrow a(C')$. GOAL-SIM requires every concrete goal state to map to an abstract goal state. Induction transports an entire concrete run to an abstract run.

Theorem 1. Refutation soundness

Under STEP-SIM and GOAL-SIM, if no abstract goal is reachable from $a(G, W_0)$, no declared run of that pack reaches ϕ_goal . Proof idea: any concrete success would transport to an abstract success, contradicting the failed abstract reachability query. This is a pack-relative implication, not a claim that extraction captured the user's intent or that real tools implement their declared effects.

Theorems 2-3. Tolerance and capability monotonicity

Tolerance soundness: when every fine-model transition is also a coarse-model transition and the goal predicate is unchanged, a coarse-model refutation also refutes the fine model. Adding abstract transitions may introduce spurious witnesses or lose a refutation; it cannot invalidate a refutation obtained in that over-approximation.

Capability monotonicity: if Γ is included in Γ' and the old guards/effects, goal and initial context are unchanged, every existing witness remains available. The proof lifts step-relation inclusion to reachability. It does not cover replacing an existing tool with different semantics or assuming an agent will choose the witness.

Theorems 4-5. Operational correspondence

Subject reduction: a step of a well-typed session has a matching protocol step, and the residual session remains typed. Session fidelity gives the converse matching step from protocol to session. The paper assumes commuting effects for world-changing actions by different participants; communication-only steps satisfy this condition automatically. These are results for the formal direct judgment, not evidence that the Python projection/conformance implementation is mechanically equivalent to it.

Proposition 1. Why admission remains conditional

An abstract witness need not correspond to a successful concrete execution when abstraction forgets a relevant distinction. Thus ACHIEVABLE is not a runtime guarantee. Intent review addresses the source-to-pack gap; runtime verification addresses the declared-effect-to-actual-payload gap. Neither obligation disappears because a structural check passed.

S10. Termination boundary and proof coverage

Theorem 6. Termination of widened search

For fixed finitely many roles, regular protocols represented as finite tail-recursive control graphs, decidable state logic and logic-definable effects, the source paper's widened symbolic search terminates (original Appendix D.5). It decides the widened abstract judgment: IMPOSSIBLE transfers to exact pack reachability under the simulation hypotheses, whereas ACHIEVABLE may be spurious.

The argument combines finite residual control, finitely many Boolean valuations, and numeric widening. On a control-graph back edge, the reference procedure forgets accumulated numeric constraints by replacing them with true. A visited-set search then explores a finite abstract space; each edge uses a decidable SMT query. This is the paper's reference-search argument, not a new proof of all executable paths.

Proposition 2. Limits of exact reachability

Exact reachability with unbounded integer counters can already encode a two-counter machine under fixed participants. Separately, the paper's dynamic-topology extension allows an unbounded role population, fresh per-role state and dynamic channels; its undecidability argument encodes communicating finite-state machines with unbounded queues. Numeric widening alone does not make that control space finite. These are specific expressiveness boundaries, not a claim that all agent autonomy is undecidable.

Table S4. Proof coverage reported in the source paper and its formalization.

Result	Mechanized versus paper-level scope
Refutation and tolerance	Coq theorem schemas, axiom-free; refutation is parameterized by step and goal simulation.
Capability monotonicity	Step-inclusion transport mechanized; capability inclusion implies step inclusion by the paper's operational argument.
Subject reduction / session fidelity	Single-label communication with full interleaving mechanized; head-move action case mechanized; general action interleaving proved on paper with side conditions.
Symbolic instantiation / termination / undecidability	Paper-level arguments, not mechanized proofs of the executable symbolic checker.

Formalization locations are `proof\SkillAchievability.v` (reachability schemas and `FlightInstance`), `proof\DirectTyping.v` (direct-typing core and `HandoffInstance`), and `proof\DirectTypingSR.v` (operational correspondence). The source reports these scoped developments as axiom-free. This adaptation reports their scope; it does not claim a new proof run or an end-to-end verified compiler.

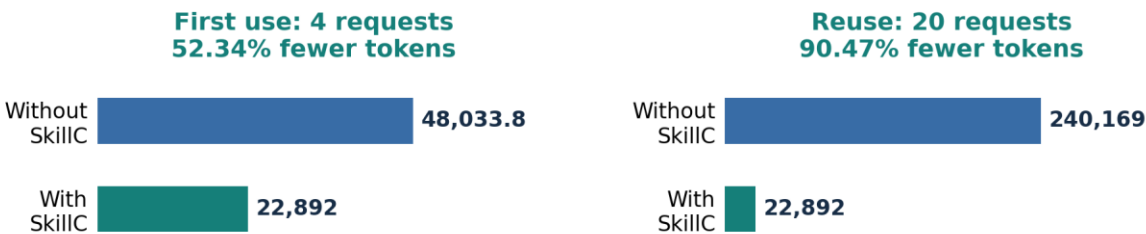
What the implementation may conclude

The executable planner is narrower than the formal model: one-role finite-Boolean synthesis with explicit resource limits. A timeout or unsupported fragment is an abstention, not evidence of impossibility. Likewise, a rejected declared protocol does not refute every alternative plan; goal-wide rejection needs an independent capability-context certificate. The formal results motivate these boundaries, while the benchmark separately measures their behavior on authored tasks.

S11. Corrected with/without-SkillC reuse experiment

Our September 21 corrected experiment compared five restricted skill profiles, each with five trials capped at eight model turns. GPT-5.4 chose actions through real Azure OpenAI calls; the tool effects were simulated with guard-aware Boolean/numeric semantics. These measurements answer a repeated-use question, not the live-backend question in S4. Compaction was measured previously and reused once per unchanged skill.

Corrected repeated-use benchmark | Four independently impossible tasks



Real API usage; simulated tool actions. Compaction paid once per unchanged skill.

First-use baseline = sum of five-trial means; reuse baseline = all 20 measured attempts.

Figure S9. The same measured compaction cost is amortized across repeated requests. Both panels exclude the achievable web-testing case; neither measures production prevalence.

Independently impossible task	With SkillC: once	Without: 5 trials	Outcomes
mcp-builder	4,338	94,542	0/5 successes
frontend-design	4,039	59,791	0/5 successes
slack-gif-creator	8,737	12,182	0/5 successes
algorithmic-art	5,778	73,654	0/5 successes
Total	22,892	240,169	0/20 successes

The independent action-level oracle identified these four tasks as impossible in their declared capability models before cost selection. All 20 attempts ended blocked. Their provider ledgers reconcile to 240,169 tokens; the four historical compaction costs reconcile to 22,892. Avoided model usage is 217,277 tokens (90.47%). The first-use baseline sums each task's five-trial mean: 48,033.8 tokens, giving 52.34% fewer tokens after paying the same compaction cost.

Quality boundary. The fifth task, web-app testing, is excluded because its goal has an alternative legal plan. Its ungated trials include two verified simulated successes, yet the historical SkillC policy rejected the declared protocol. Preventing those successes is not a benefit. Across all five tasks the ledger records 92 fresh model calls and 260,242 tokens; the four-task savings claim must not be applied to that mixed set.

Provenance: runs\20260921_110234Z_real_rejection_benchmark contains results.json, accuracy.json, runtime_trials.jsonl, calls.jsonl and verification.json. Compaction records are reused from runs\20260916_132708Z_real_rejection_benchmark. The correction report documents the repaired semantics and supersedes the older 60.8%/92.2% savings claims. No new model calls were made for this manuscript.

S12. Earlier experiments: larger numbers, different claims

Our earlier 134-run experiment used Haiku/Sonnet agent executions. Of 66 runs whose configurations were checker-refuted, 20 nevertheless produced verified results on small inputs. The remaining 46 unsuccessful runs consumed 15,841,106 measured runtime tokens. This is a retrospective failure-only subset, not evidence that rejecting all 66 runs preserves task completion.

Historical subset	With SkillC	Without SkillC	Reduction
PDF: compaction once, 4 failures	22,011	157,454	86.02%
XLSX: compaction once, 4 failures	32,028	209,766	84.73%
PDF + XLSX: 8 failures	54,039	367,220	85.28%
PDF + XLSX: first-use means	54,039	91,805	41.14%
All 46 failures: mixed evidence	209,195	15,841,106	98.68%

PDF and XLSX have recorded compaction calls and recorded runtime trials: both cost sides are measured. The 85.28% comparison pays compaction once per skill and reuses it over four failed executions per skill. It is a cost accounting comparison, not a paired quality-preserving test of a gated agent. For one expected execution of each skill, the same costs yield 41.14%, using the four-trial runtime means.

Why 98.68% is not interchangeable with 46.74%. The larger 98.68% figure uses 209,195 compaction tokens, comprising 54,039 measured and 155,156 estimated from skill length, against 15,841,106 measured runtime tokens. Four already-formal configurations require no natural-language compaction. The 46 failures are selected retrospectively and exclude all 20 verified successes in refuted configurations; it is not a fully measured, end-to-end savings claim.

The corrected 90.47% result selects independently impossible cases, uses real API usage with simulated tools, and amortizes compaction across five uses. The 46.74% result instead measures real backend operations across achievable and impossible profiles, with all 28 achievable tasks completed with and without the gate. Different tasks, models, baselines and reuse policies explain the different percentages; we report them separately rather than selecting the largest as a universal result.

The August pilot is also retained: at one use, 20,064 compaction tokens exceeded the raw 1,919 credited avoidable tokens, and three of six achievable cases were falsely rejected. It did not establish a positive savings claim. Its result is under runs\20260827_113833Z_token_economics_pilot.

Historical provenance: commit dbb855d1e51c550ce7cdabaf4624e0c2de0325ec, paper\WIP\results\usefulness_runs.jsonl, usefulness.json, compaction_runs.jsonl and token_economics.json. These were read without switching branches; docs\TOKEN_ECONOMICS_EVIDENCE_GUIDE.md explains the measured/estimated distinction; docs\BENCHMARK_CORRECTION_20260921.md documents superseded simulator results. These paths identify our own experiment artifacts, not related-work citations.